

Tony Astolfi
January 16, 2026

TurtleKV

A Dynamically Tunable Key/Value Storage Engine

Why Build A New Key/Value Storage Engine?

Goal of TurtleKV: *simplify and reduce the **cost of (re-)tuning** the storage engine to an application or workload, while offering best-in-class performance on modern hardware*

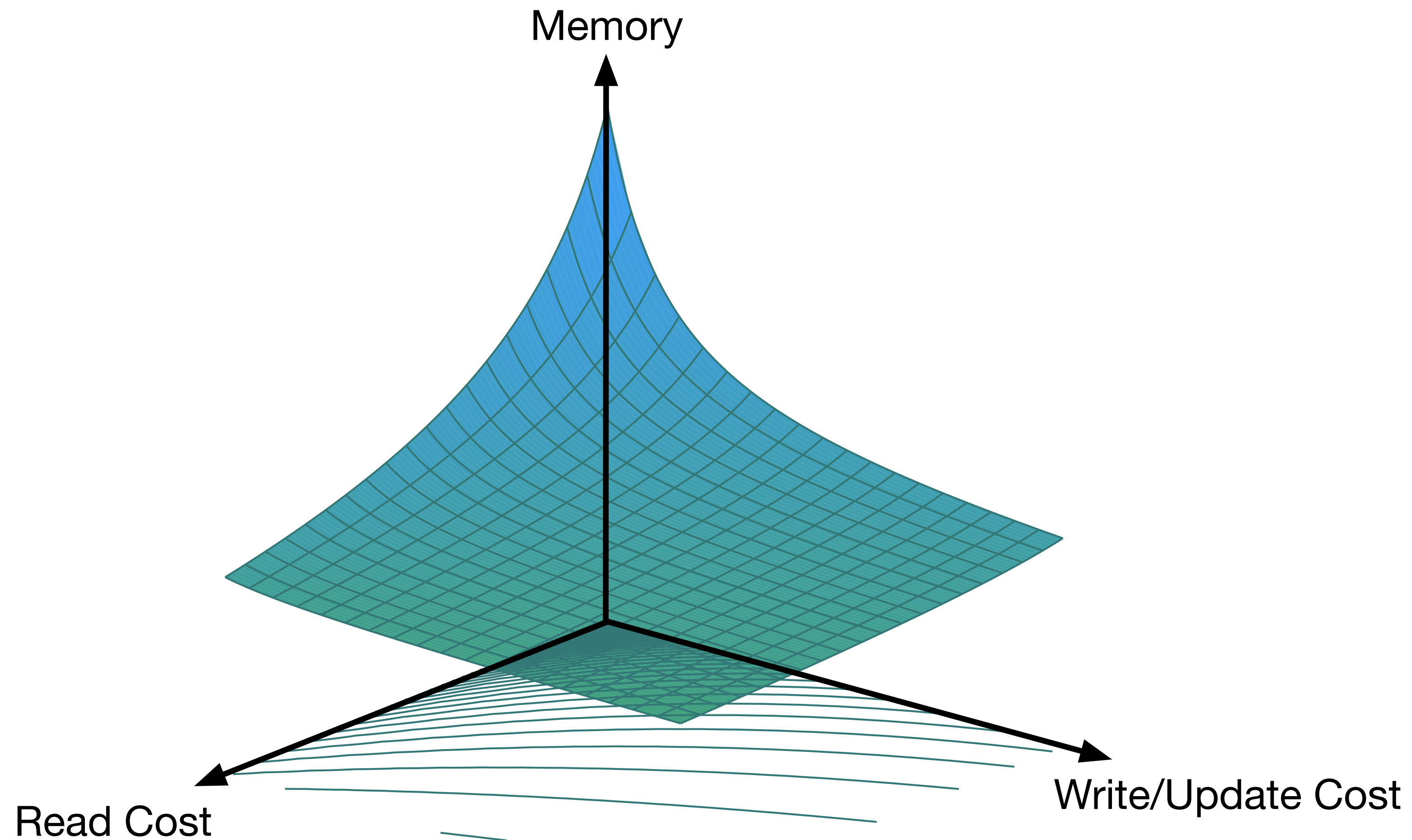
Are Trade-offs Inevitable?

- (Almost Certainly) Yes.
- Optimal Reads, Updates, Memory Footprint: Pick 2
 - G. S. Brodal and R. Fagerberg, “Lower Bounds for External Memory Dictionaries,” SODA, vol. 3, 2003
 - Z. Wei, K. Yi, and Q. Zhang, “Dynamic external hashing: the limit of buffering”. SPAA, 2009
 - M. Athanassoulis et al., “Designing Access Methods: The RUM Conjecture,” EDBT, 2016
- *Research Question:* Are a multitude of storage engine designs inevitable..?



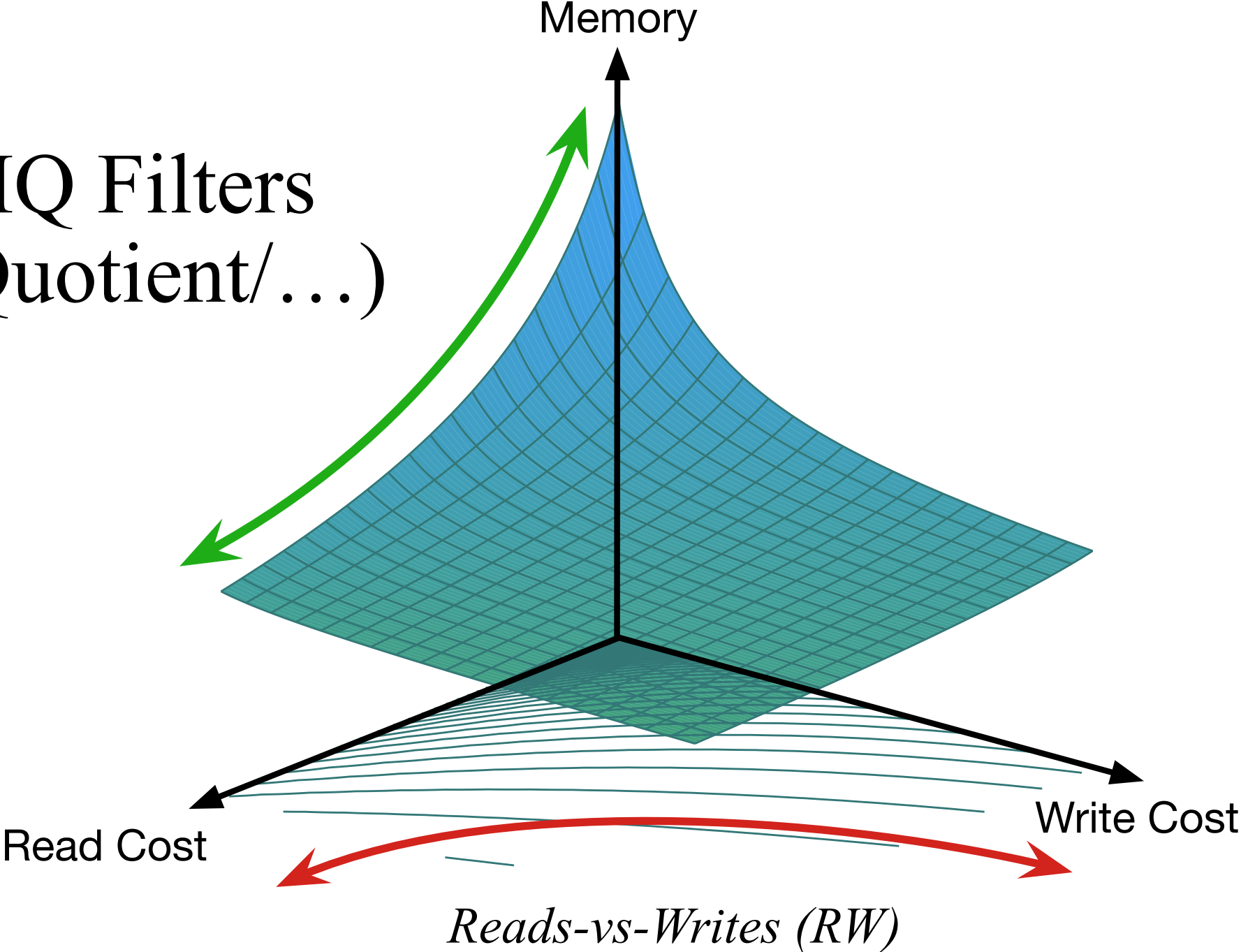
What exactly is a “three-way” trade-off?

Committing to a certain level of one resource puts the remaining two in zero-sum

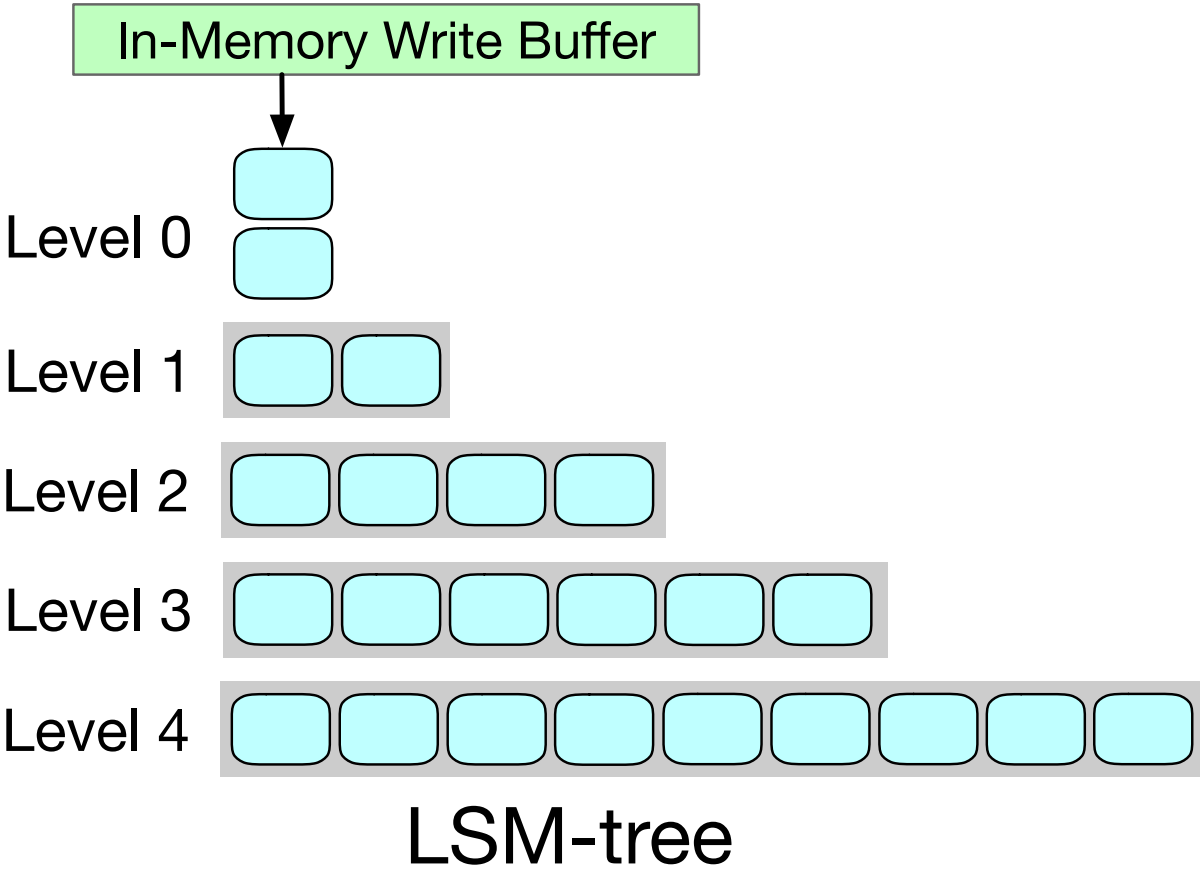


RUM Through the Lens of Data Structures

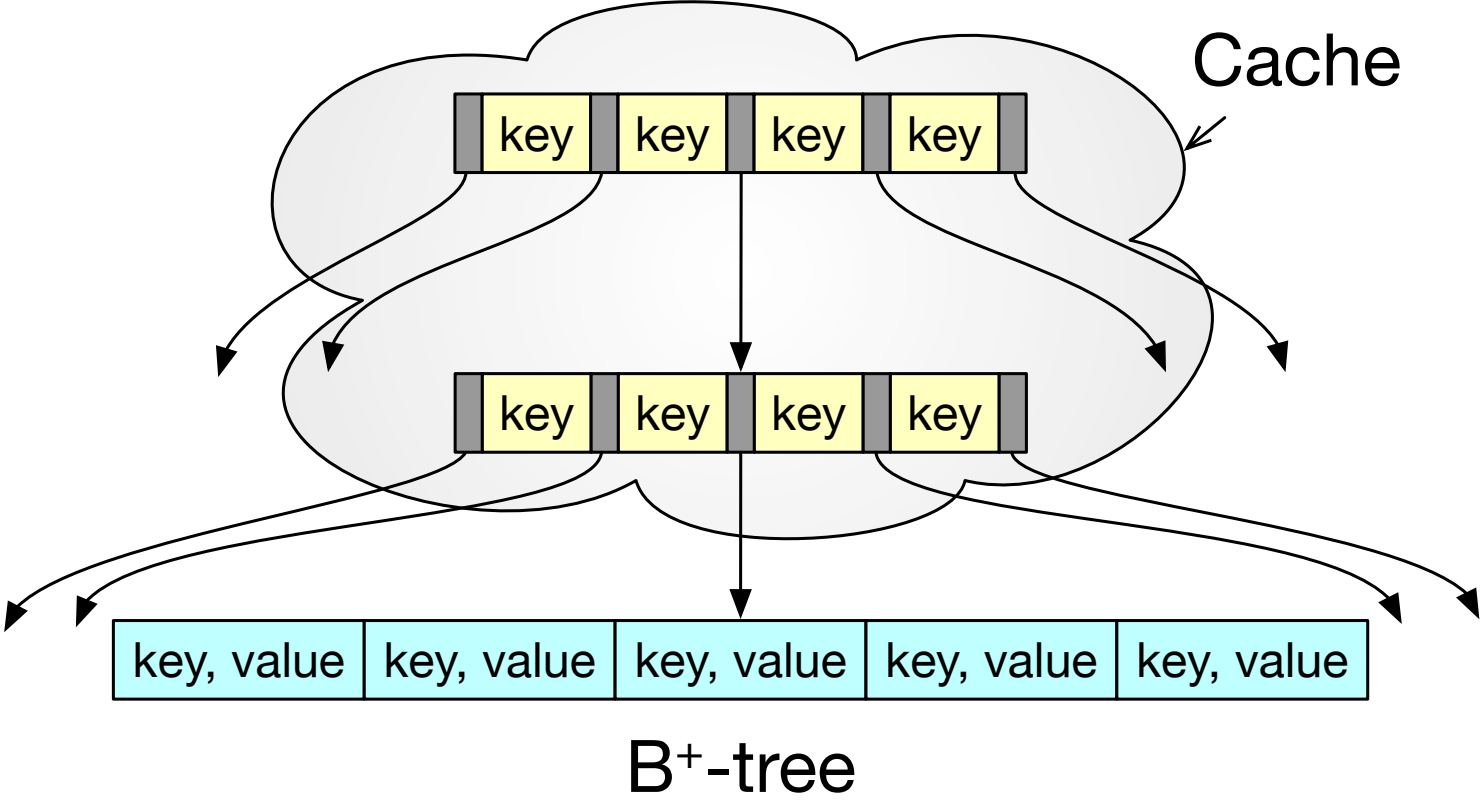
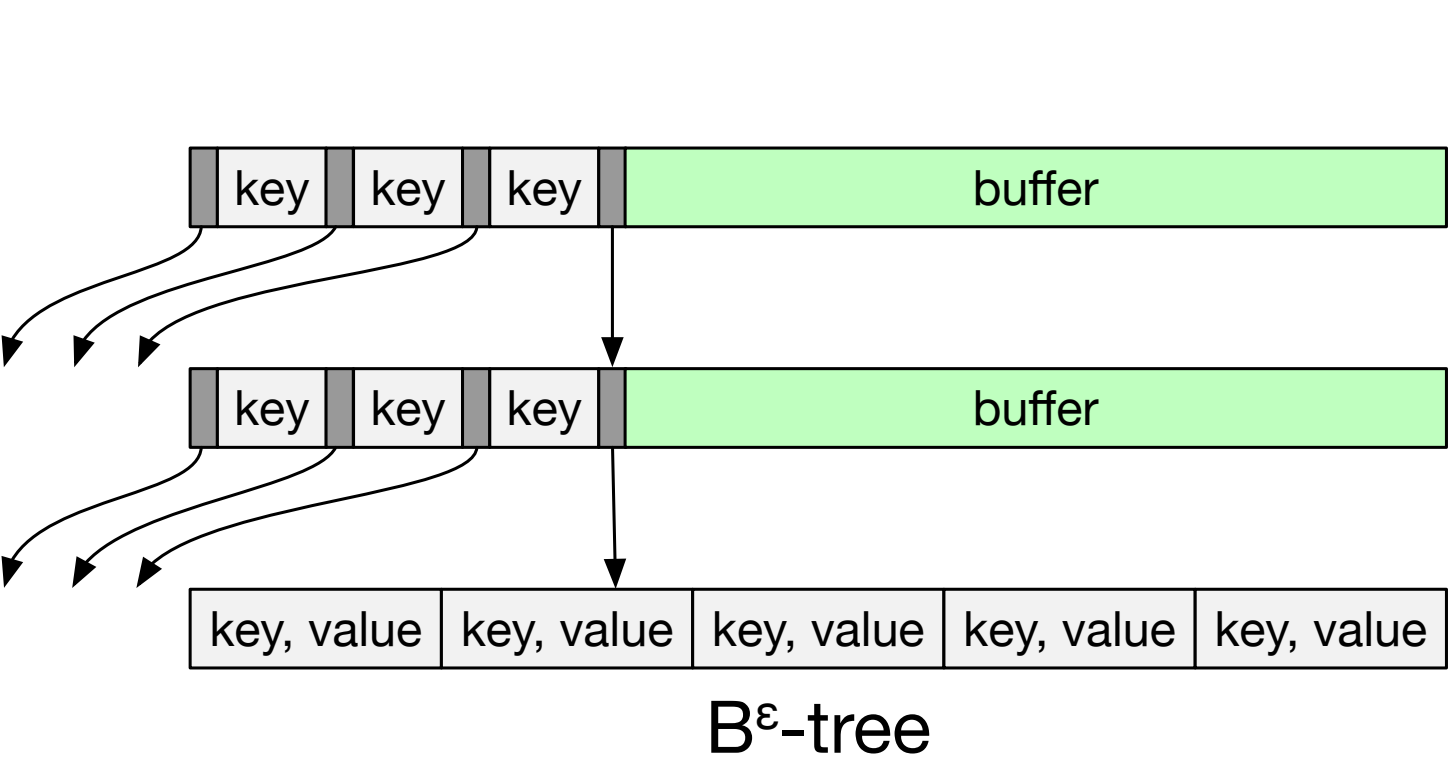
RM: Caching, AMQ Filters
(Bloom/Cuckoo/Quotient/...)



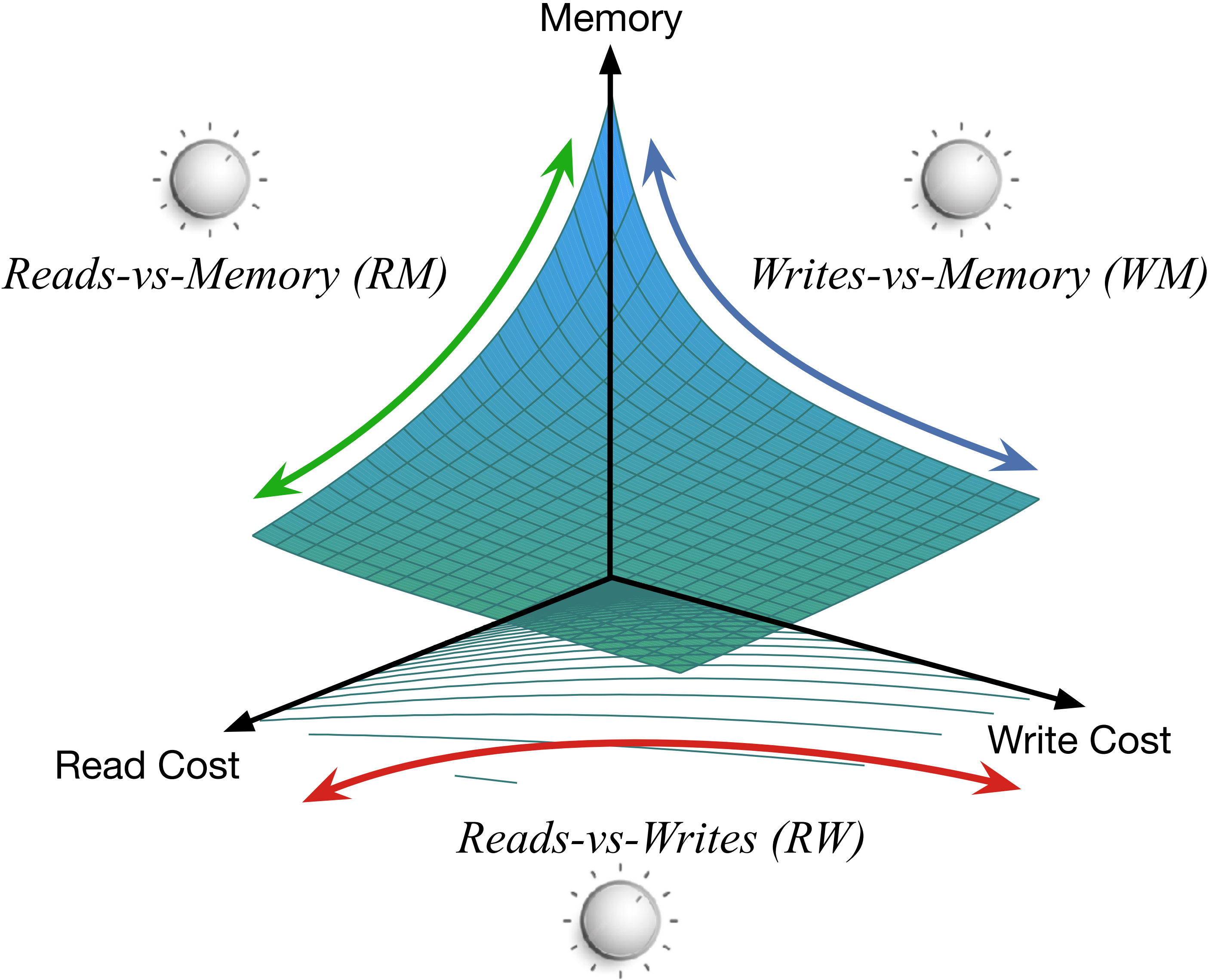
Write-optimized



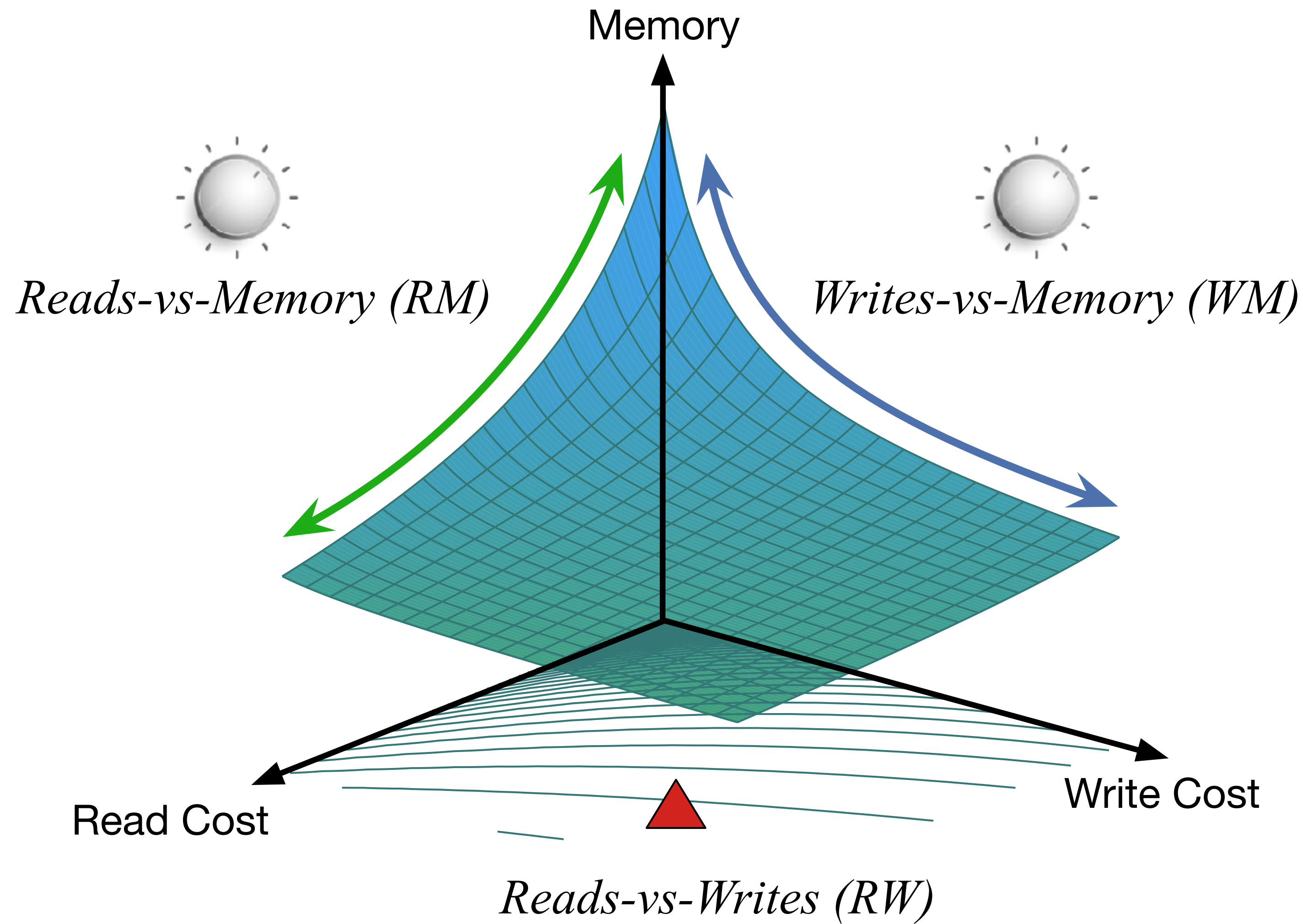
Read-optimized



What Tuning Knobs Should We Choose?



Key Insight: Dynamic Tuning (RM + WM)

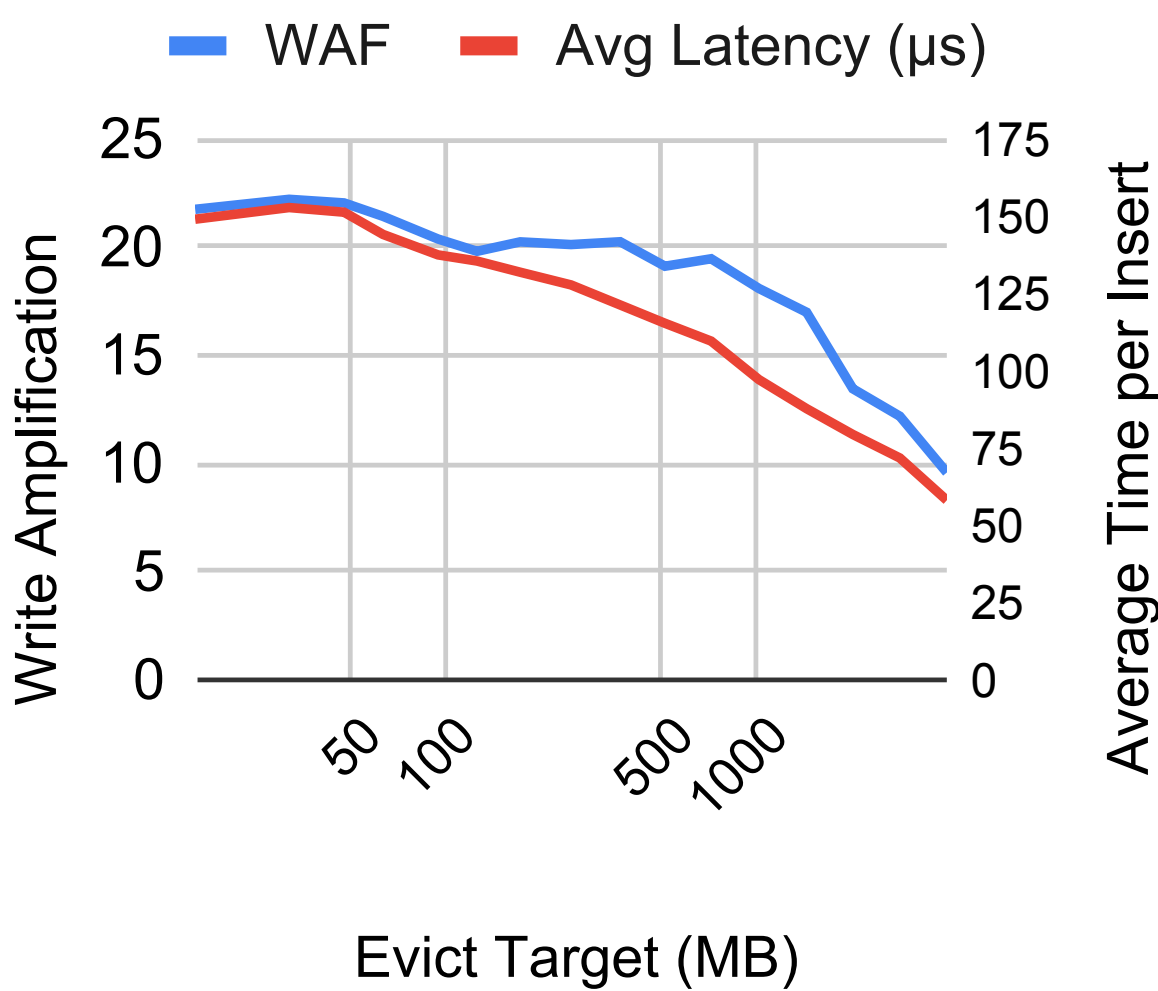


Pay memory to optimize reads or writes, *only when you need the optimization!*

Support for Write-vs-Memory (WM) Tuning

WiredTiger (B⁺-Tree)

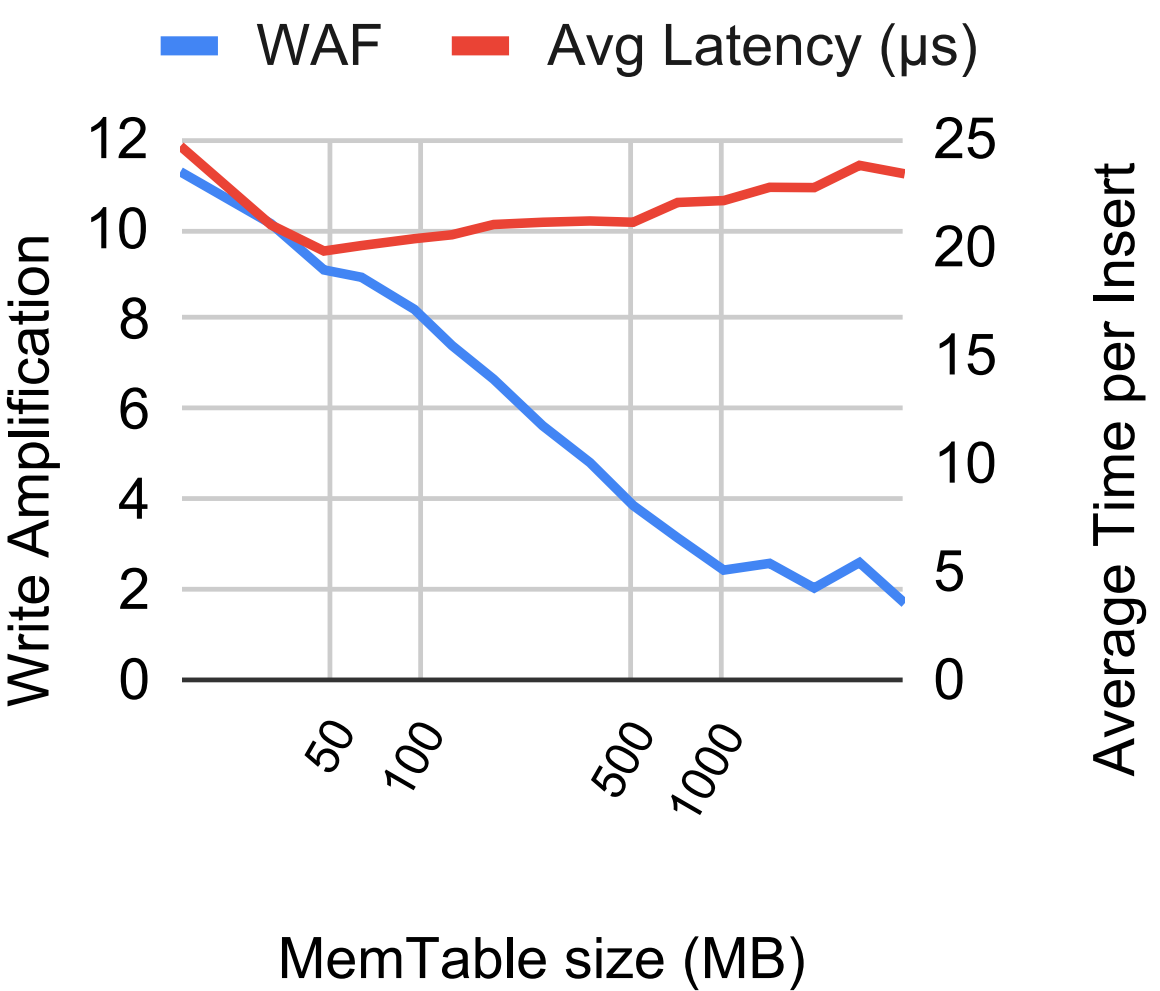
eviction dirty target/trigger



Data structure allows efficient, scalable write-buffering

RocksDB (LSM)

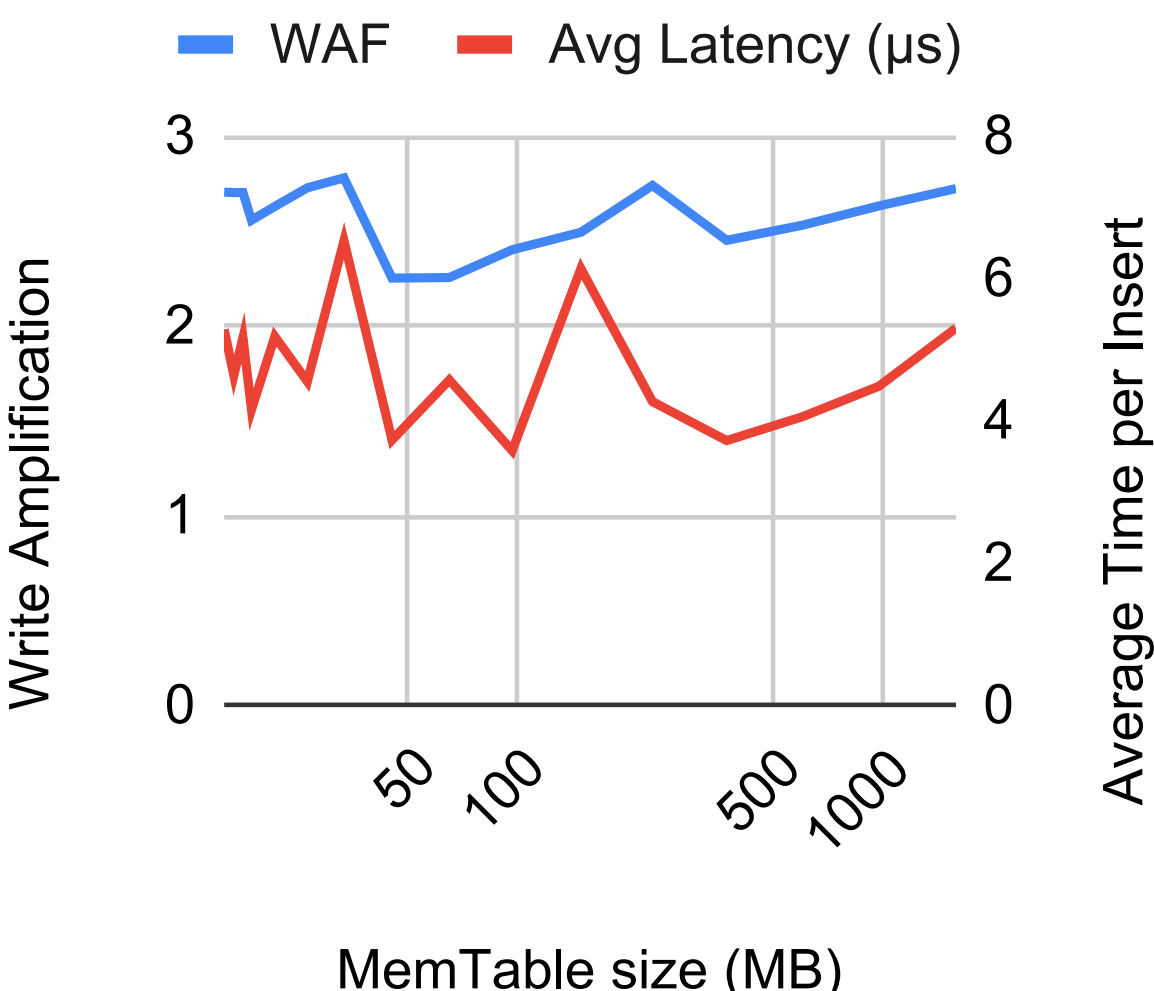
write_buffer_size



No in-memory bottlenecks that negate WAF reduction

SplinterDB (STB^ε-Tree)

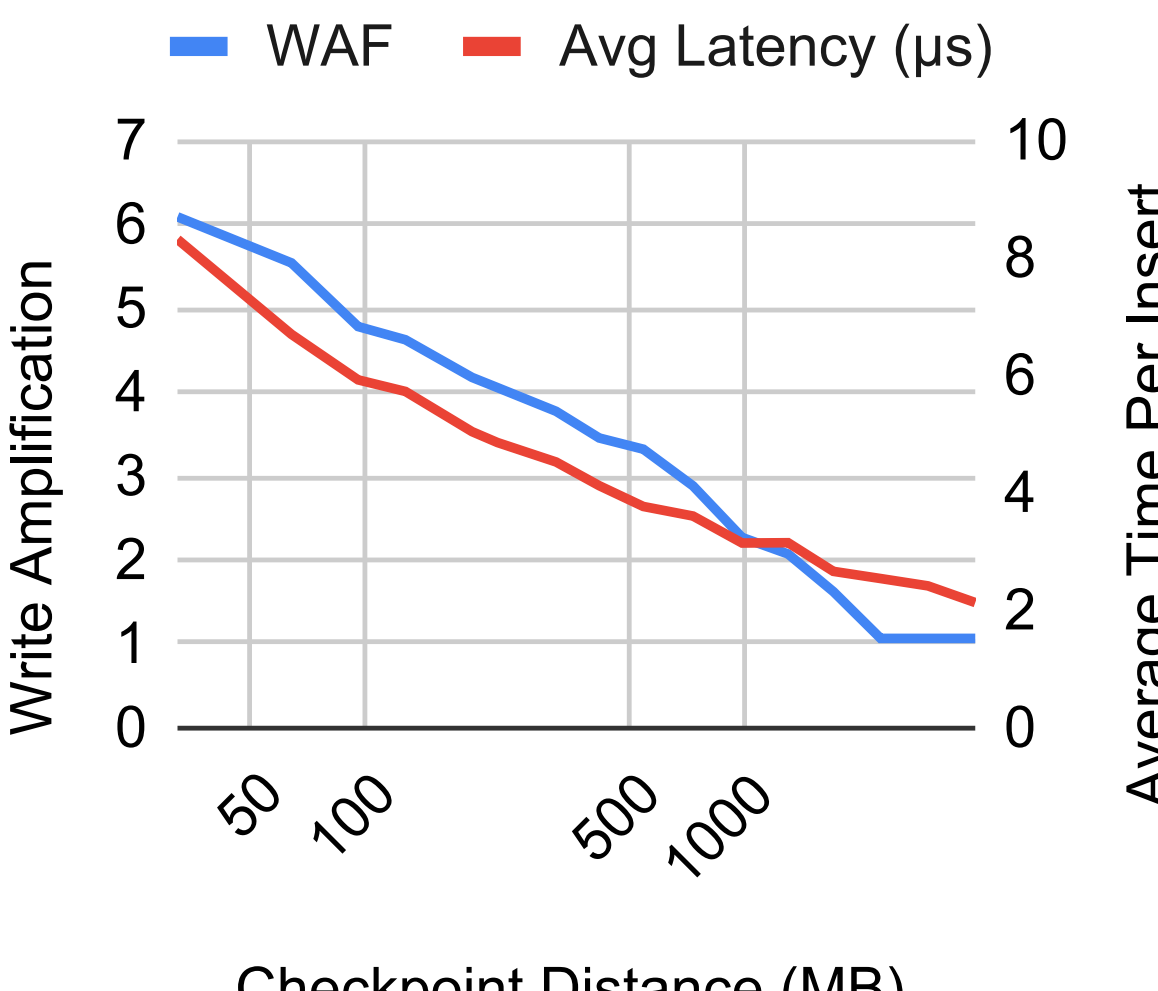
memtable size?



Write-Memory (WM) has a dedicated tuning knob

TurtleKV (TurtleTree)

checkpoint distance

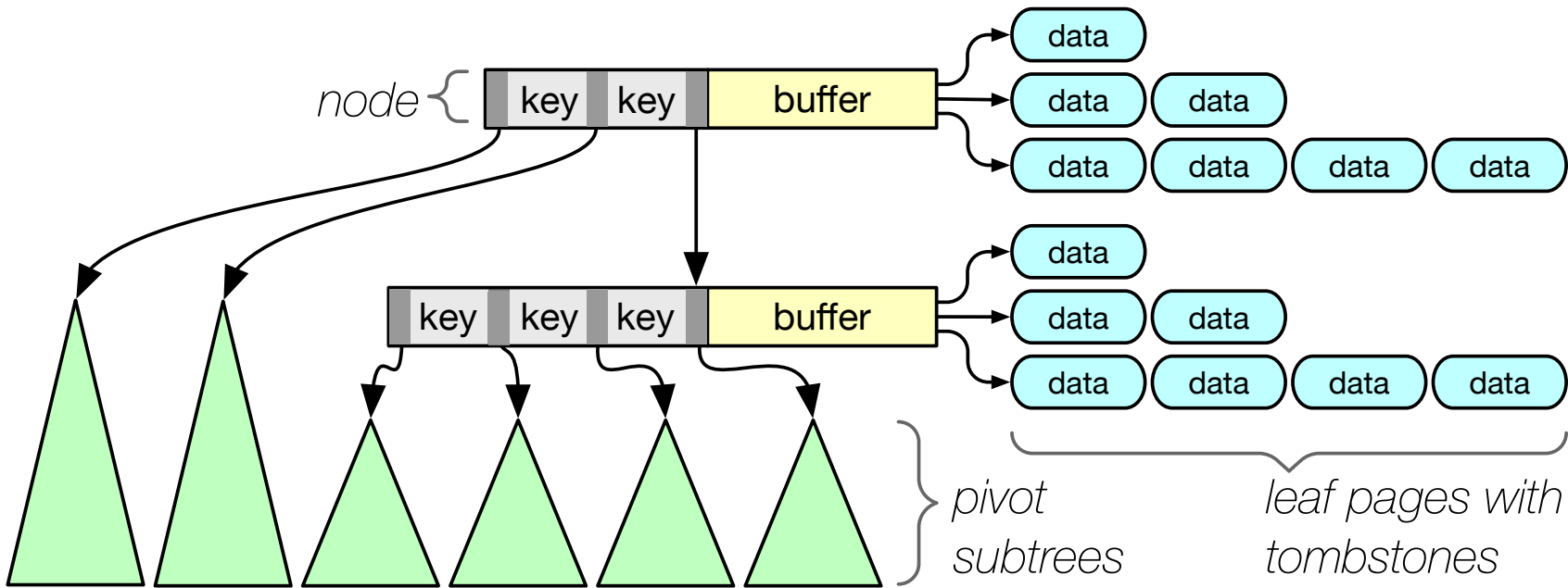


✓ *Dynamic Tuning*

TurtleKV: Overview of Key Ideas

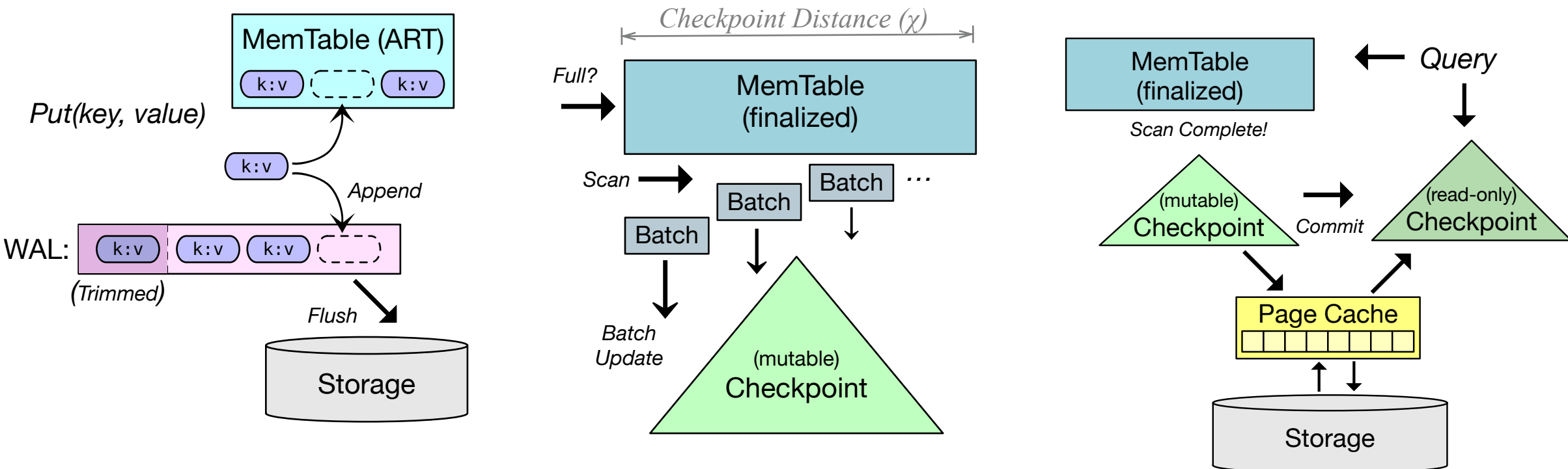
1. Data Structure: TurtleTree

- Like SplinterDB’s STB ϵ -Tree, but balanced instead of write-optimized
- Cache-/scan-friendly



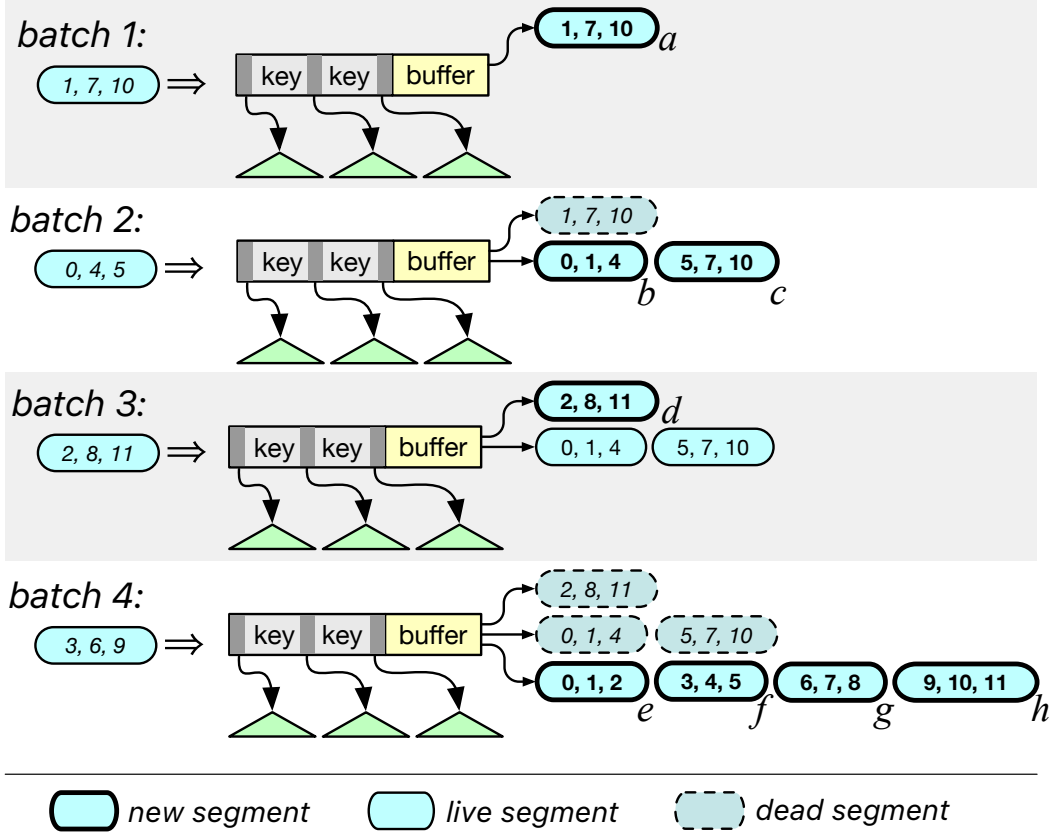
2. System Design:

- “Big” MemTables
- Zero-copy compactions in-memory
- Multi-core, data-parallel merging



3. WM Tuning Knob: Checkpoint Distance

- Scale Invariant

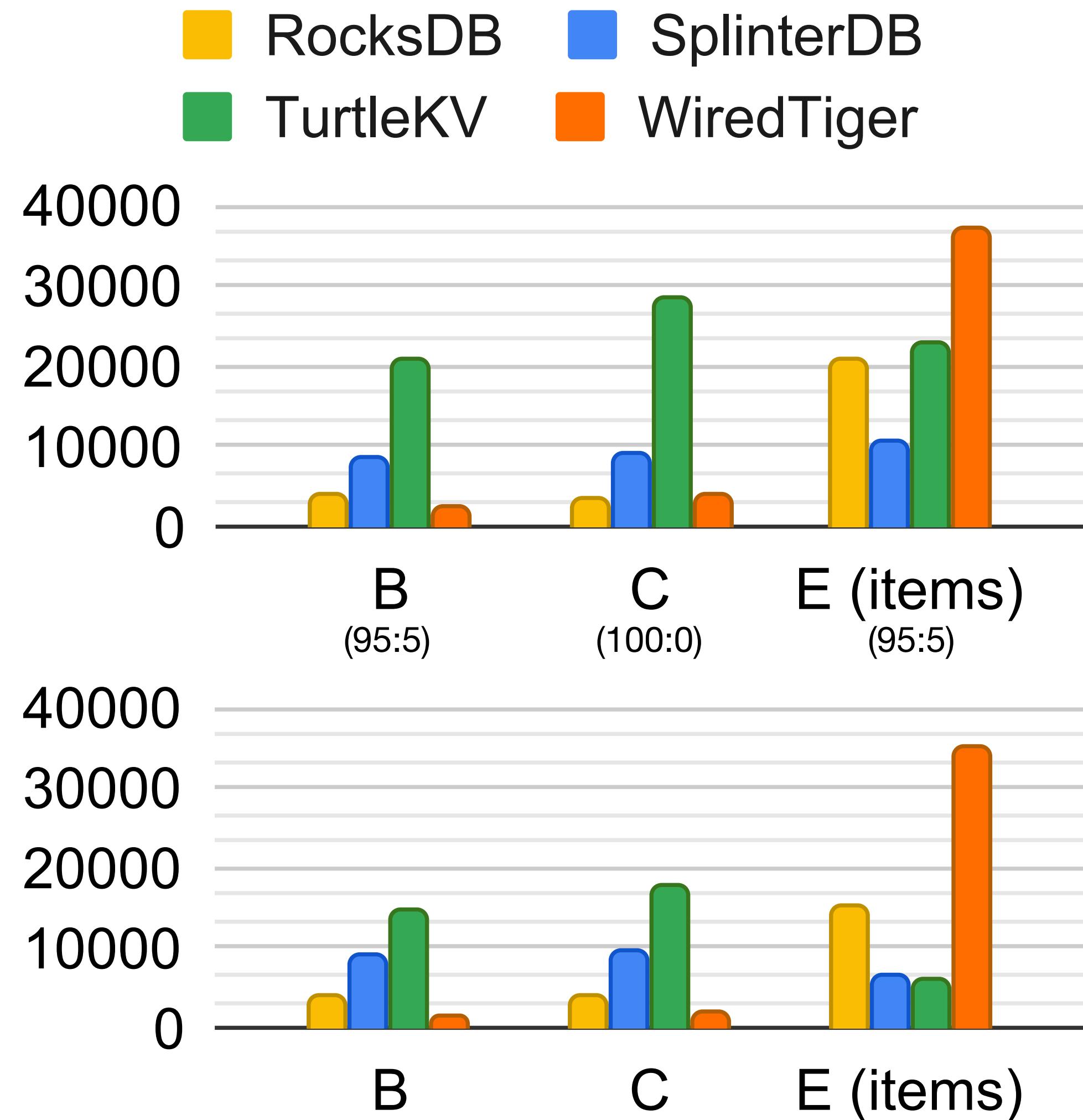
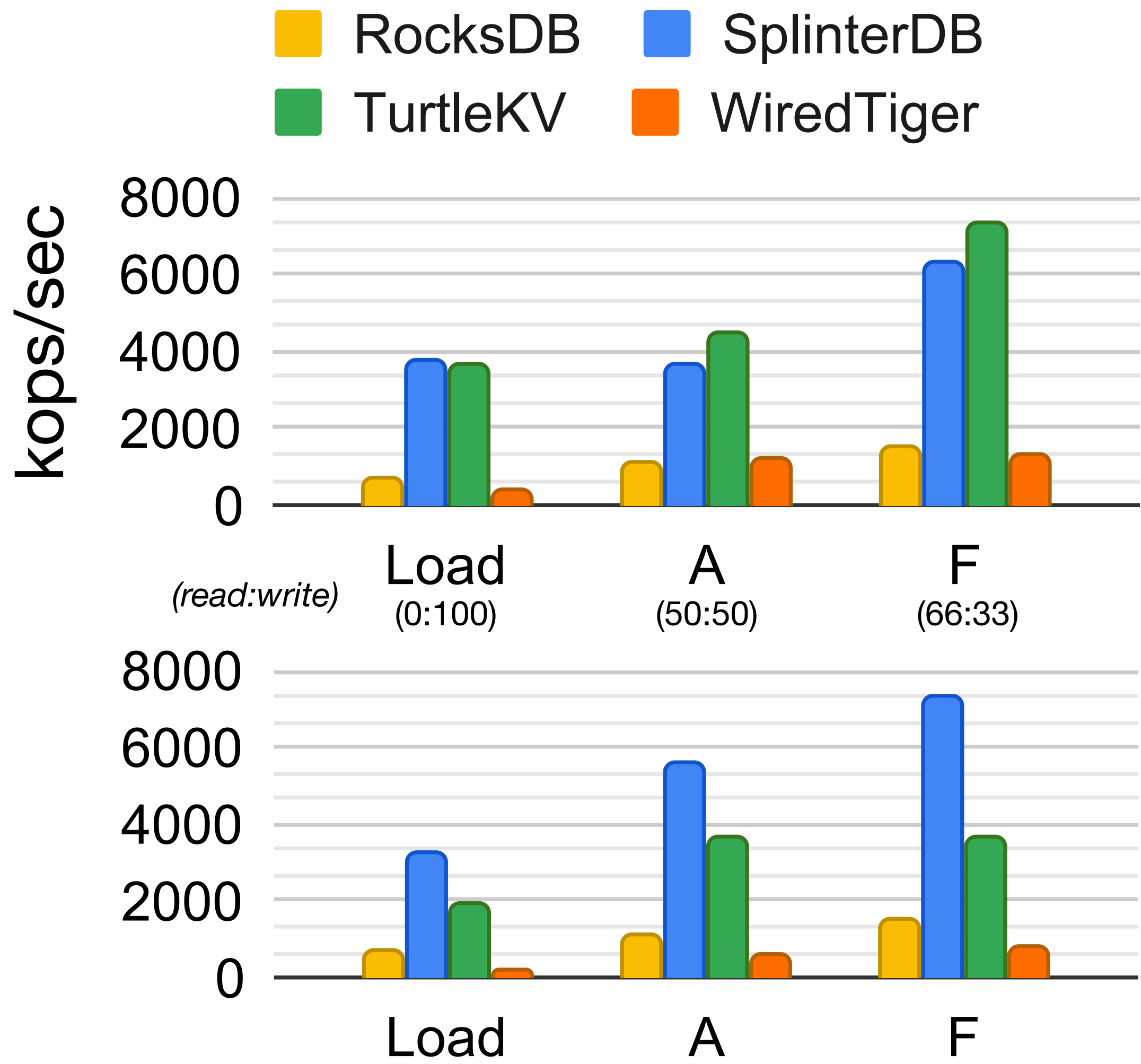


Page	Lifetime				Written if ..?			
	t=1	t=2	t=3	t=4...	$\chi=1$	$\chi=2$	$\chi=3$	$\chi=4$
a	1, 7, 10				✓	○	○	○
b	0, 1, 4				✓	✓	✓	○
c	5, 7, 10				✓	✓	✓	○
d	2, 8, 11				✓	○	○	○
e	0, 1, 2				✓	✓	✓	✓
f	3, 4, 5				✓	✓	✓	✓
g	6, 7, 8				✓	✓	✓	✓
h	9, 10, 11				✓	✓	✓	✓

Yahoo Cloud Serving Benchmark (YCSB)

(more memory)

(less memory)



Summary

- Trade-offs are inevitable; multiple key/value stores are not!
- RUM = 3-way trade-off = 2D surface in 3D resource space = {RM, RW, WM}
- RW is a hassle; RM is great; WM has been neglected
- TurtleKV offers strong performance, ability to dynamically tune via (RM, WM)
- Key Ideas: TurtleTrees, Optimized Update Pipeline, Checkpoint Distance
- Source code: https://github.com/mathworks/turtle_kv
- Paper (preprint): <https://arxiv.org/pdf/2509.10714>

Thank you!

Questions?

